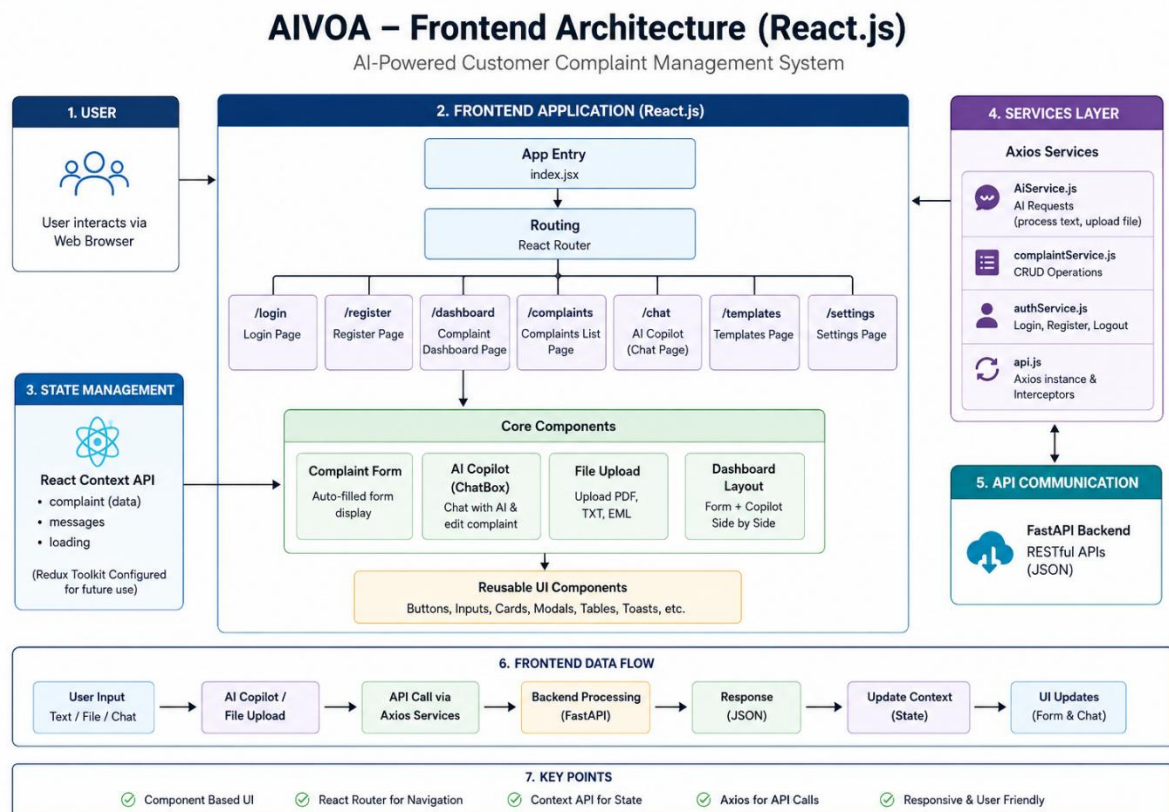


AIVOA PROJECT

The frontend architecture of aivoa



1. WHAT IS REACT.JS AND WHY IS IT USED FOR THE FRONTEND OF THIS PROJECT?

React.js is a JavaScript library used to build user interfaces using reusable components. In this project, React is used to create the frontend screens and manage their UI state. It also allows the application to update specific parts of the interface without reloading the entire page.

2. WHAT IS THE PURPOSE OF REACT ROUTER IN THIS PROJECT?

React Router is used to manage navigation between different frontend pages. It allows routes such as login, registration, dashboard, profile, and other application pages to be handled on the client side. It also supports protected routes for authenticated users.

3. HOW IS AUTHENTICATION HANDLED BETWEEN THE REACT FRONTEND AND FASTAPI BACKEND?

The React frontend sends login credentials to the FastAPI authentication API.
After successful authentication, the backend provides an access token.
The frontend stores the token and sends it with subsequent protected API requests.

4. WHAT IS THE ROLE OF AUTHCONTEXT IN THE FRONTEND?

AuthContext provides authentication information to different React components.
It manages information such as the current authentication state and access token.
This avoids passing authentication data manually through multiple components.

5. WHY IS THE ACCESS TOKEN STORED IN LOCALSTORAGE AND HOW IS IT USED FOR API REQUESTS?

The access token is stored in localStorage so it can remain available when the user navigates between pages or reloads the application.
Axios reads the token from localStorage and adds it to the Authorization header of protected API requests.
The backend then uses the token to authenticate the request.

6. WHAT IS THE PURPOSE OF AXIOS INTERCEPTORS IN THIS PROJECT?

Axios interceptors allow common request and response processing to be handled centrally.
The request interceptor adds the access token to API requests.
The response interceptor can detect authentication failures and handle token refresh logic.

7. HOW DOES THE FRONTEND HANDLE A 401 UNAUTHORIZED RESPONSE AND REFRESH THE ACCESS TOKEN?

When the backend returns 401 Unauthorized because the access token has expired, the Axios response interceptor detects it.
It calls the refresh token logic to obtain a new access token.
The original request can then be retried using the refreshed token.

8. HOW IS THE RESUME OR COMPLAINT DATA FETCHED FROM THE BACKEND AND DISPLAYED IN REACT COMPONENTS?

React components call backend APIs through the Axios service layer.
The returned data is stored in component state or application state.
The component then uses that state to render the required information in the UI.

9. HOW ARE REUSABLE COMPONENTS USED TO AVOID DUPLICATING FRONTEND CODE?

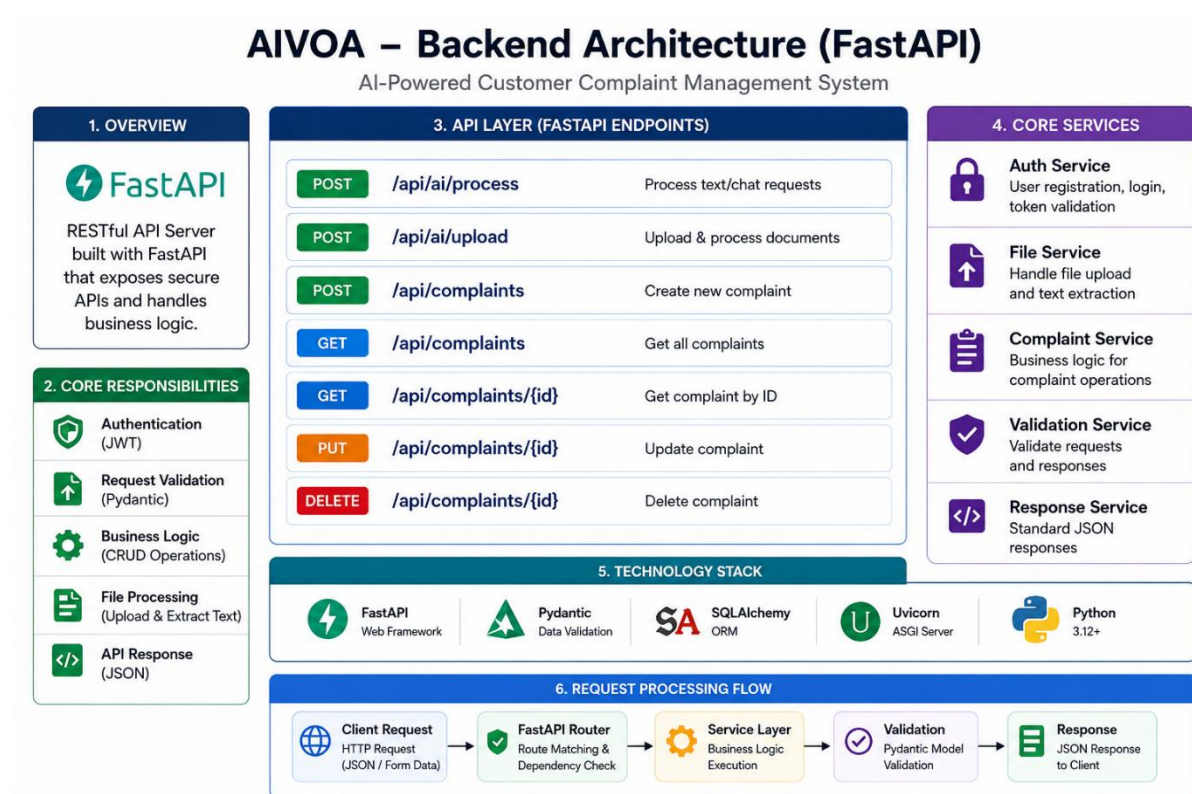
Common UI elements are created as reusable React components instead of writing the same code repeatedly.
For example, a common layout, navigation component, form component, or upload component can be reused across pages.
This makes the frontend easier to maintain and update.

10. HOW WOULD YOU PROTECT PRIVATE PAGES SO THAT UNAUTHENTICATED USERS CANNOT ACCESS THEM DIRECTLY?

I would use a ProtectedRoute component around routes that require authentication. It checks whether a valid authentication state or access token exists before rendering the page.

If the user is not authenticated, the application redirects them to the login page.

The backend architecture of the aivoo



Part 1 of 3: AI Layer

This is the most important part for explaining the intelligent part of the project in an interview.

1. backend/app/ai/graph.py

Purpose:

Defines and connects the LangGraph workflow.

Function:

`route_intent(state: ComplaintState)`

What it does:

Checks state["intent"] and decides which AI node should execute.

Possible outputs:

`log_complaint`

`edit_complaint`

`document_extraction`

Hinglish:

Ye function basically AI router hai. User ne kya request kiya hai uske according decide karta hai ki complaint log karni hai, existing complaint edit karni hai, ya document se complaint extract karni hai.

Important graph nodes registered in this file:

`detect_intent`

`log_complaint`

`edit_complaint`

`document_extraction`

`risk_assessment`

Final object:

`graph = graph_builder.compile()`

Hinglish:

Ye complete LangGraph ko compile karke executable graph banata hai.

2. backend/app/ai/nodes.py

Ye AI layer ki main processing file hai.

Function:

`_parse_and_validate(content: str)`

What it does:

Parses the LLM response as JSON, removes markdown code fences if present, normalizes some risk fields and validates the response using `AIComplaintResponse`.

Hinglish:

LLM kabhi kabhi JSON ko code block ke andar return kar sakta hai. Ye function response ko clean karke proper JSON banata hai aur schema ke according validate karta hai.

Function:

```
detect_intent_node(state: ComplaintState)
```

What it does:

Uses the LLM to classify the user's request.

Possible intents:

log_complaint

edit_complaint

document_extraction

Stores the result in:

```
state["intent"]
```

Hinglish:

Ye sabse pehle user ki request ko samajhta hai. Example, user bolta hai "batch number change karo", to AI ise edit_complaint intent samajh sakta hai.

Function:

```
log_complaint_node(state: ComplaintState)
```

What it does:

Uses the LLM to extract complaint information from user input.

It stores:

```
state["complaint"]
```

```
state["risk_assessment"]
```

```
state["assistant_response"]
```

Hinglish:

User complaint deta hai, ye function us complaint se structured information nikalta hai, jaise customer, product, batch, quantity, complaint type etc. Uske baad risk assessment bhi store karta hai.

Function:

```
edit_complaint_node(state: ComplaintState)
```

What it does:

Updates only the complaint fields mentioned by the user.

It preserves existing complaint information.

It also identifies changed fields.

Hinglish:

Agar existing complaint me user sirf batch number change karta hai, to ye function sirf batch number update karta hai. Baaki existing data ko delete nahi karta.

Example:

Existing:

Batch = ABC123

Quantity = 50

User:

"Batch number ABC456 kar do."

Result:

Batch = ABC456

Quantity = 50

Hinglish:

Matlab partial update karta hai, complete complaint ko overwrite nahi karta.

Function:

```
risk_assessment_node(state: ComplaintState)
```

What it does:

Calls:

```
generate_risk_assessment(state["complaint"])
```

Then stores the result in:

```
state["risk_assessment"]
```

Hinglish:

Complaint data ready hone ke baad ye function risk analysis karwata hai aur final risk assessment state me save karta hai.

Function:

```
document_extraction_node(state: ComplaintState)
```

What it does:

Takes uploaded document content and asks the LLM to extract complaint information.

Then:

Stores extracted complaint.

Generates risk assessment.

Sets assistant response.

Hinglish:

Agar user PDF, TXT ya EML document deta hai, to ye function document ke content se complaint details extract karta hai aur uske baad risk assessment generate karta hai.

3. backend/app/ai/risk.py

Function:

`generate_risk_assessment(complaint: dict)`

What it does:

Analyzes complaint data using the LLM and generates:

`severity`

`next_action`

`justification`

`complaint_summary`

`root_cause_recommendation`

`capa_recommendation`

`completeness_check`

`duplicate_complaint_check`

Hinglish:

Ye project ka actual risk analysis function hai. Complaint ko dekhkar AI decide karta hai ki issue kitna serious hai, next action kya hona chahiye, possible root cause kya investigate karna chahiye aur CAPA recommendation kya honi chahiye.

4. backend/app/ai/llm.py

Object:

`base_llm`

Configuration:

`ChatGroq`

Model:

`llama-3.1-8b-instant`

Temperature:

`0`

Purpose:

Creates the LLM instance used by the AI layer.

Hinglish:

Ye file actual Groq LLM connection/configuration banati hai. Baaki AI functions isi base_llm ko use karke LLM ko request bhejte hain.

5. backend/app/ai/prompts.py

Constants:

INTENT_CLASSIFIER_SYSTEM_PROMPT

Purpose:

Defines instructions for intent classification.

Hinglish:

AI ko batata hai ki user ki request ko log_complaint, edit_complaint ya document_extraction me classify karna hai.

LOG_COMPLAINT_SYSTEM_PROMPT

Purpose:

Defines instructions for:

Complaint extraction

Complaint editing

Risk assessment

Validation rules

JSON output structure

Hinglish:

Ye AI ko complete rules deta hai ki complaint se kya information extract karni hai, missing information ke saath kya karna hai, edit kaise karna hai aur risk assessment ka output kis format me dena hai.

6. backend/app/ai/state.py

Class:

ComplaintState

Purpose:

Defines the shared state used by LangGraph.

Fields:

user_message

uploaded_document

intent

complaint

risk_assessment

assistant_response

error

Hinglish:

Ye LangGraph ka shared data container hai. Ek AI node jo information generate karta hai, next node usi state se information read karta hai.

Example:

detect_intent_node sets intent.

log_complaint_node sets complaint.

risk_assessment_node sets risk_assessment.

7. backend/app/ai/output_schema.py

Class:

ComplaintExtractionOutput

Field:

complaint: dict

Purpose:

Defines the output structure for complaint extraction.

Hinglish:

Ye ensure karta hai ki complaint extraction ka output expected structure ke andar aaye.

Interview me Part 1 ko aise explain karna:

"Sir, I have separated the AI layer into multiple responsibilities. graph.py manages the LangGraph workflow and routing. nodes.py contains the actual AI processing nodes for intent detection, complaint extraction, editing and document extraction. risk.py handles risk assessment. llm.py configures the Groq LLM, while prompts.py contains the system instructions and state.py maintains the shared LangGraph state. Finally, the schema files validate the structured AI output."

Hinglish meaning:

"AI layer ko maine responsibility ke according divide kiya hai. Graph workflow manage karta hai, nodes actual AI operations perform karte hain, risk.py risk assessment karta hai, llm.py Groq model configure karta hai, prompts.py AI instructions rakhta hai aur state.py different AI nodes ke beech shared data maintain karta hai."

part 2 of 3: Backend Core Layer

This part covers the API, database, models, validation, configuration and document utilities.

1. backend/app/main.py

What is main.py?

main.py is the entry point of the FastAPI application. It creates the application and connects the backend API routers.

Hinglish:

main.py backend ka starting point hai. FastAPI application yahin se start hoti hai aur API routes ko application ke saath connect karta hai.

Exact functions:

The actual main.py source code is not available, so I will not invent function names.

Follow-up questions:

What is the purpose of main.py?

Why are routers included in FastAPI?

2. backend/app/api/ai.py

What is ai.py?

ai.py contains the API endpoints responsible for AI related requests such as processing text or chat input and processing uploaded documents.

Hinglish:

Ye file frontend se aane wali AI requests receive karti hai aur unhe AI layer tak bhejne ka API interface provide karti hai.

API responsibilities:

POST /api/ai/process

Processes text or chat requests.

POST /api/ai/upload

Processes uploaded complaint documents.

Exact function names:

The actual ai.py source code is not available, so exact function names cannot be verified.

Follow-up questions:

Why separate AI APIs from complaint APIs?

What is the difference between text processing and file processing?

3. backend/app/api/complaints.py

What is complaints.py?

complaints.py contains the REST API endpoints for complaint management operations.

Main operations:

Create complaint.

Get all complaints.

Get complaint by ID.

Update complaint.

Delete complaint.

Hinglish:

Ye file complaint ke CRUD operations ke APIs handle karti hai. Frontend complaint create, view, update ya delete karna chahe to requests isi API layer ke through aati hain.

Endpoints:

POST /api/complaints

GET /api/complaints

GET /api/complaints/{id}

PUT /api/complaints/{id}

DELETE /api/complaints/{id}

Exact function names:

The actual complaints.py source code is not available, so I will not guess them.

Follow-up questions:

What is CRUD?

Why use separate API routes for complaints?

What is the purpose of {id}?

4. backend/app/core/config.py

What is config.py?

config.py manages application configuration and environment variables.

Examples:

Groq API key.

Database configuration.

Other application settings.

Hinglish:

Ye file configuration ko centralize karti hai. API keys aur environment based settings ko directly code me hardcode karne ke bajay configuration se load kiya jata hai.

Follow-up questions:

Why should API keys not be hardcoded?

What is the purpose of environment variables?

5. `backend/app/db/database.py`

What is `database.py`?

`database.py` contains the database configuration and SQLAlchemy database setup.

Hinglish:

Ye file application aur database ke beech database connection setup karne ka kaam karti hai.

Main responsibility:

Database connection configuration.

SQLAlchemy setup.

Database engine configuration.

Exact function names:

The source code is not available, so exact functions cannot be verified.

Follow-up questions:

Why use SQLAlchemy?

What is a database engine?

6. `backend/app/db/session.py`

What is `session.py`?

`session.py` manages SQLAlchemy database sessions used for database operations.

Hinglish:

Database ke saath actual query operations karne ke liye session chahiye hota hai. Ye file database sessions ko manage karti hai.

Example responsibility:

Create database session.

Provide session for queries.

Close session after operation.

Exact function names:

The source code is not available, so I will not invent them.

Follow-up questions:

What is a database session?

Why should a database session be closed?

7. `backend/app/db/dependencies.py`

What is dependencies.py?

dependencies.py provides database related dependencies to FastAPI endpoints.

Hinglish:

FastAPI dependency injection ka use karke API functions ko required database session provide karne ka kaam ye file karti hai.

Conceptually:

API endpoint

|

v

Database dependency

|

v

Database session

Follow-up questions:

What is dependency injection in FastAPI?

Why use dependencies instead of creating a session inside every API?

8. backend/app/models/complaint.py

What is complaint.py?

complaint.py defines the SQLAlchemy Complaint model that represents complaint data in the database.

It maps Python objects to database records.

Hinglish:

Ye database ka complaint table ka Python representation hai. SQLAlchemy ORM ke through Python object ko database table se map karta hai.

Important complaint data includes:

complaint_number

complaint_source

customer_name

product_name

product_strength_grade

batch_lot_number

manufacturing_date

expiry_date

quantity_affected

complaint_type

complaint_date

detailed_complaint_description

initial_severity

priority

created_at

updated_at

Follow-up questions:

What is ORM?

Why use SQLAlchemy models?

What is the difference between a model and a Pydantic schema?

9. backend/app/schemas/complaint.py

What is complaint.py inside schemas?

This file defines the Pydantic schemas used for complaint data validation and request or response structures.

Hinglish:

models database ke structure ko represent karta hai, while schemas API ke data ko validate karte hain.

Simple difference:

Model = Database structure.

Schema = API data validation.

Follow-up questions:

Why use Pydantic?

Why not use SQLAlchemy model directly for API validation?

10. backend/app/schemas/risk_assessment.py

What is risk_assessment.py?

This file defines the schema used to validate risk assessment data.

Risk assessment contains information such as:

severity

next_action

justification

complaint_summary

root_cause_recommendation

capa_recommendation

completeness_check

duplicate_complaint_check

Hinglish:

AI risk assessment generate karta hai, aur ye schema ensure karta hai ki risk assessment expected structure me aaye.

Follow-up questions:

Why validate AI generated output?

What happens if the AI returns an incorrect structure?

11. backend/app/schemas/ai_response.py

Class:

ComplaintOutput

What does ComplaintOutput do?

ComplaintOutput defines the complete AI response structure containing:

complaint

risk_assessment

Hinglish:

Ye complete AI response ka validation structure hai. Isme complaint data aur risk assessment dono expected format me validate hote hain.

Follow-up questions:

Why validate LLM output?

Why keep complaint and risk_assessment separately?

12. backend/app/utils/document_parser.py

What is document_parser.py?

document_parser.py is responsible for extracting readable content from uploaded complaint documents.

Supported document types in the project include:

PDF

TXT

EML

Hinglish:

User agar complaint PDF, TXT ya email file upload karta hai, to document parser us file ka

readable text extract karne ke liye use hota hai. Ye extracted content AI layer ko diya ja sakta hai.

Follow-up questions:

Why do we need a document parser?

Why convert documents into text before sending them to the LLM?

Exact function names:

The actual source code is not available, so I will not guess the parser function names.

Part 2 interview answer:

"Sir, the backend core layer is separated into API, database, model, schema and utility responsibilities. The api package exposes the REST endpoints, the models package defines SQLAlchemy database models, the schemas package validates request and response data using Pydantic, the db package manages database connections and sessions, core manages configuration, and utils handles document parsing. This separation keeps the backend modular and makes the code easier to maintain and extend."

Hinglish:

"Backend ko maine separate responsibilities me divide kiya hai. API layer requests handle karti hai, models database structure represent karte hain, schemas data validation karte hain, db database sessions manage karta hai, core configuration handle karta hai aur utils document processing karta hai."

Part 3 of 3: Supporting Backend Files

1. backend/app/ai/schemas.py

What is schemas.py inside ai?

It defines the structured data schema used by the AI layer for complaint information and validation.

Hinglish:

Ye AI ke output ko expected complaint structure me define karta hai, taaki LLM ka response predictable rahe.

Follow-up questions:

Why do we need a schema for LLM output?

What happens if the LLM returns invalid data?

2. backend/app/ai/output_schema.py

Class:

ComplaintExtractionOutput

What does it do?

It defines the expected output structure for complaint extraction.

Field:

complaint: dict

Hinglish:

LLM se complaint extraction ke time expected output structure define karta hai.

Follow-up question:

Why use structured output from an LLM?

3. backend/app/ai/state.py

Class:

ComplaintState

What does it do?

It stores the shared data used by different LangGraph nodes.

Important fields:

user_message

uploaded_document

intent

complaint

risk_assessment

assistant_response

error

Hinglish:

Ye AI workflow ka shared state hai. Ek node jo data generate karta hai, next node wahi state se data use karta hai.

Follow-up question:

Why use shared state in LangGraph?

4. backend/app/ai/llm.py

Object:

base_llm

What does it do?

Creates the Groq Chat LLM used by the AI processing functions.

Configuration:

ChatGroq

Model: llama-3.1-8b-instant

Temperature: 0

Hinglish:

Ye file Groq LLM ka configuration create karti hai. AI nodes isi LLM instance ko use karte hain.

Follow-up question:

Why is temperature set to 0?

5. backend/app/ai/prompts.py

Constant:

INTENT_CLASSIFIER_SYSTEM_PROMPT

What does it do?

Provides instructions for classifying user intent.

Constant:

LOG_COMPLAINT_SYSTEM_PROMPT

What does it do?

Provides instructions for complaint extraction, editing and risk assessment.

Hinglish:

Prompts.py AI ko rules aur instructions deta hai ki input ko kaise understand aur process karna hai.

Follow-up question:

Why keep prompts in a separate file?

6. backend/app/utils/document_parser.py

Purpose:

Extracts text from uploaded complaint documents such as PDF, TXT and EML.

Hinglish:

Ye uploaded document ko readable text me convert karta hai, jise AI layer process kar sakti hai.

Follow-up question:

Why extract text before sending a document to the LLM?

7. backend/app/uploads

Purpose:

Stores uploaded complaint files.

Hinglish:

User ke uploaded documents ko temporarily store karne ke liye use hota hai.

It is a folder, so it does not contain functions.

8. backend/.env

Purpose:

Stores environment-specific configuration such as API keys and database credentials.

Hinglish:

Sensitive configuration ko source code se separate rakhne ke liye .env use hoti hai.

Follow-up question:

Why should .env not be committed to Git?

Final interview summary:

AI files:

graph.py = LangGraph workflow

nodes.py = AI processing functions

risk.py = Risk assessment

llm.py = Groq LLM configuration

prompts.py = LLM instructions

state.py = Shared AI state

schemas.py = AI data structures

output_schema.py = Structured extraction output

Backend files:

main.py = FastAPI application

api = REST endpoints

core = Configuration

db = Database connection and sessions

models = Database models

schemas = API validation

utils = Document parsing

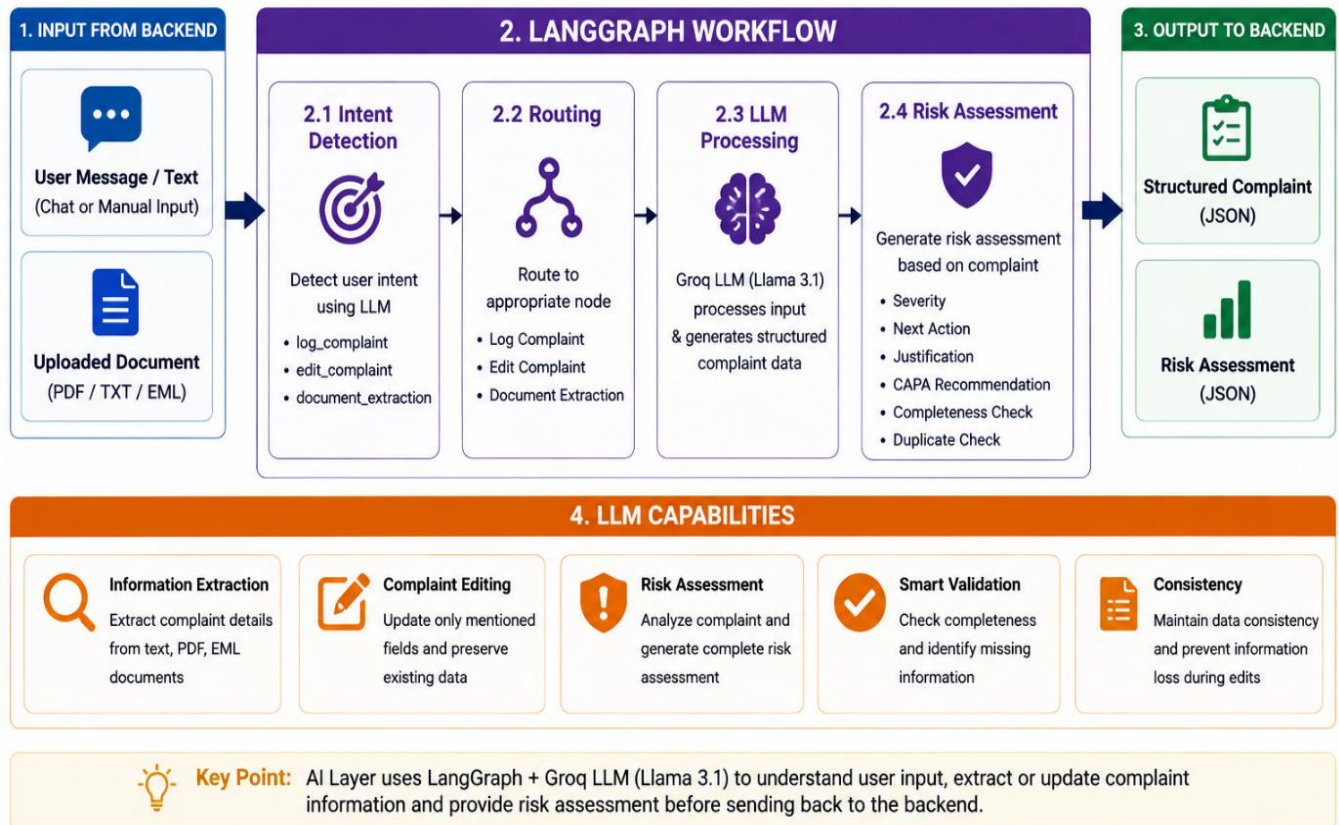
uploads = Uploaded files storage

This is the complete three-part backend explanation based on the project material available.

The ai layer of the aivoa

AIVOA – AI Layer Architecture

AI Layer Handles Intent, Extraction, Editing & Risk Assessment



AI Layer

1. backend/app/ai/graph.py

route_intent(state: ComplaintState)

Routes the request according to the detected intent: log_complaint, edit_complaint, or document_extraction.

graph

Builds and compiles the LangGraph workflow.

Hinglish:

Ye file AI workflow ko connect aur route karti hai.

2. backend/app/ai/nodes.py

_parse_and_validate(content: str)

Parses, cleans and validates the LLM JSON response.

`detect_intent_node(state: ComplaintState)`

Detects whether the user wants to log, edit, or extract a complaint.

`log_complaint_node(state: ComplaintState)`

Extracts structured complaint data from user input and stores the complaint and risk assessment.

`edit_complaint_node(state: ComplaintState)`

Updates only the complaint fields changed by the user and preserves existing data.

`risk_assessment_node(state: ComplaintState)`

Generates and stores the risk assessment for the complaint.

`document_extraction_node(state: ComplaintState)`

Extracts complaint information from uploaded document content and generates its risk assessment.

Hinglish:

Ye file AI ke actual processing functions contain karti hai.

3. `backend/app/ai/risk.py`

`generate_risk_assessment(complaint: dict)`

Generates severity, next action, justification, complaint summary, root cause recommendation, CAPA recommendation, completeness check and duplicate complaint assessment.

Hinglish:

Ye complaint ki seriousness analyze karke quality and safety related recommendations generate karta hai.

4. `backend/app/ai/llm.py`

`base_llm`

Creates the Groq LLM using llama-3.1-8b-instant with temperature 0.

Hinglish:

Ye AI layer ke liye actual LLM instance configure karta hai.

5. `backend/app/ai/prompts.py`

`INTENT_CLASSIFIER_SYSTEM_PROMPT`

Defines rules for intent classification.

`LOG_COMPLAINT_SYSTEM_PROMPT`

Defines rules for complaint extraction, editing and risk assessment.

Hinglish:

Ye file LLM ko exact instructions aur output rules provide karti hai.

6. `backend/app/ai/state.py`

ComplaintState

Maintains shared AI workflow data such as `user_message`, `intent`, `complaint`, `risk_assessment` and `assistant_response`.

Hinglish:

Ye different AI nodes ke beech common data store karta hai.

7. `backend/app/ai/output_schema.py`

ComplaintExtractionOutput

Defines the structured output for complaint extraction.

Hinglish:

Ye complaint extraction ke expected output format ko define karta hai.

8. `backend/app/ai/schemas.py`

AI related schema definitions are maintained here for structured AI data handling.

Hinglish:

Ye AI layer ke structured data ko define karne ke liye use hota hai.

Exact class names are not available in the provided source, so I will not guess them.

Interview one line:

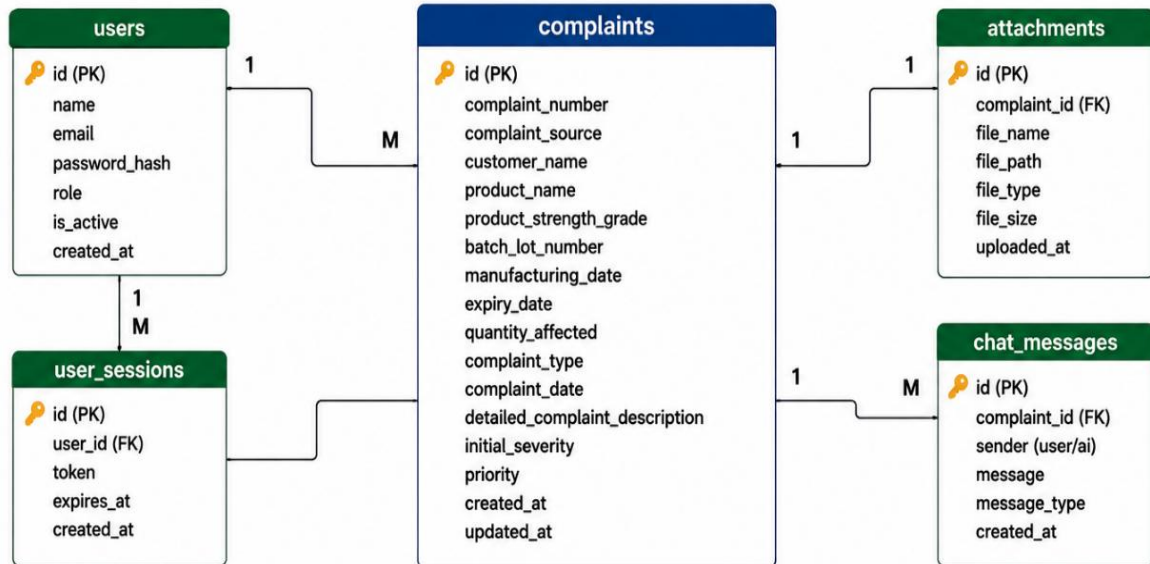
"The AI layer uses LangGraph for orchestration, Groq LLM for reasoning, `nodes.py` for AI operations, `risk.py` for risk assessment, `prompts.py` for instructions, `state.py` for shared state, and `schemas` for structured output validation."

The database architecture of the aivoa

AIVOA – DATABASE LAYER (MySQL)

Stores Complaints, Users, Attachments and Chat History

1. DATABASE SCHEMA (ER DIAGRAM)



2. KEY RELATIONSHIPS

- One User can have many Complaints
- One User can have many Sessions
- One Complaint can have many Attachments
- One Complaint can have many Chat Messages

3. TECHNOLOGIES



- Relational Database
- ACID Compliant
- InnoDB Storage Engine
- UTF8MB4 Character Set

4. PURPOSE



- Persistent storage of complaint data
- User management & authentication
- Store uploaded files metadata
- Maintain chat conversation history
- Ensure data integrity & relationships

Part 2 of 3: Database Layer

1. backend/app/db/database.py

Question: What is the purpose of database.py?

Answer:

database.py is responsible for configuring the SQLAlchemy database connection and creating the database engine used by the application.

Hinglish:

Ye file application aur MySQL database ke beech connection setup karti hai.

Interview point:

It handles database configuration and engine creation.

2. backend/app/db/session.py

Question: What is the purpose of session.py?

Answer:

session.py manages SQLAlchemy database sessions used to execute database operations safely within API requests.

Hinglish:

Ye file database ke saath actual interaction ke liye session provide karti hai. API request ke time isi session ke through data read, insert, update ya delete kiya ja sakta hai.

Interview point:

database.py creates the connection engine, while session.py provides database sessions.

3. backend/app/db/dependencies.py

Question: What is the purpose of dependencies.py?

Answer:

dependencies.py provides database-related dependencies to FastAPI endpoints, mainly allowing API functions to receive a database session through dependency injection.

Hinglish:

FastAPI endpoint ko manually database session create karne ki zarurat na pade, isliye ye file session ko dependency ke form me provide karti hai.

Interview point:

It connects FastAPI dependency injection with the database session.

4. backend/app/models/complaint.py

Question: What is the purpose of complaint.py inside models?

Answer:

complaint.py defines the SQLAlchemy Complaint model that maps the complaint table in MySQL to a Python class.

Hinglish:

Ye database table ka Python representation hai. Complaint ka database data SQLAlchemy ke through Python object ke form me handle hota hai.

The complaint model contains fields such as:

id

complaint_number

complaint_source

customer_name

product_name

product_strength_grade

batch_lot_number

manufacturing_date
expiry_date
quantity_affected
complaint_type
complaint_date
detailed_complaint_description
initial_severity
priority
created_at
updated_at

The database architecture confirms that complaints are the main complaint data table.

5. What is SQLAlchemy ORM and why is it used in this project?

Answer:

SQLAlchemy ORM allows the application to work with database tables using Python classes and objects instead of writing raw SQL for every operation.

Example:

Complaint Python class represents the complaints table.

Hinglish:

SQLAlchemy ek bridge ki tarah kaam karta hai jo Python code ko MySQL database se connect karta hai.

Interview point:

It provides object relational mapping and simplifies CRUD operations.

6. What is the difference between database.py and models/complaint.py?

Answer:

database.py handles database connection configuration, while complaint.py defines the structure of the complaint table.

Hinglish:

database.py batata hai database se connection kaise banana hai, aur complaint.py batata hai database me complaint ka data kis structure me store hoga.

7. What is a database session?

Answer:

A database session is a controlled interaction between the application and the database used to execute operations such as querying, inserting, updating and deleting records.

Hinglish:

Session ek temporary working connection jaisa hota hai jiske through application database ke records ke saath kaam karti hai.

8. Why are database dependencies used in FastAPI?

Answer:

FastAPI dependencies allow database sessions and other required resources to be automatically provided to API endpoint functions.

Hinglish:

Isse har endpoint me manually database connection manage nahi karna padta. FastAPI dependency injection ke through required session provide kar deta hai.

9. What tables are present in the database design?

Answer:

The current database design contains users, complaints, user_sessions, attachments and chat_messages.

Hinglish:

users user information rakhta hai, complaints complaint information rakhta hai, user_sessions authentication sessions rakhta hai, attachments uploaded files ka metadata rakhta hai aur chat_messages AI/user conversation store karta hai.

10. What is the relationship between users and complaints?

Answer:

One user can have many complaints, so the relationship is one-to-many.

Example:

One user can create multiple complaints.

Hinglish:

Ek user multiple complaints create kar sakta hai, isliye users aur complaints ke beech one-to-many relationship hai.

11. What is the relationship between complaints and attachments?

Answer:

One complaint can have many attachments.

Example:

A complaint can contain multiple uploaded PDF, TXT or EML files.

Hinglish:

Ek complaint ke saath multiple documents attach ho sakte hain.

12. What is the relationship between complaints and chat_messages?

Answer:

One complaint can have many chat messages.

Example:

A complaint can have multiple user and AI messages.

Hinglish:

Ek complaint ke context me user aur AI ke multiple messages store ho sakte hain.

13. What is a primary key?

Answer:

A primary key uniquely identifies each record in a database table.

Example:

The id field is the primary key of the complaints table.

Hinglish:

Primary key har database record ki unique identity hoti hai.

14. What is a foreign key?

Answer:

A foreign key connects one table with another table by referencing the primary key of another table.

Example:

attachments.complaint_id references the complaint record.

Hinglish:

Foreign key tables ke beech relationship establish karta hai.

15. What is the role of MySQL in this project?

Answer:

MySQL provides persistent relational storage for users, complaints, sessions, attachments and chat messages.

Hinglish:

MySQL application ka permanent data store hai. Application restart hone ke baad bhi complaint aur related data database me available rehta hai.

16. What is the role of SQLAlchemy in this project?

Answer:

SQLAlchemy acts as the ORM layer between the FastAPI application and MySQL database.

Hinglish:

FastAPI ke Python code ko MySQL tables ke saath easily work karne ke liye SQLAlchemy ORM use kiya gaya hai.

Important interview line:

"The database layer uses MySQL for persistent relational storage and SQLAlchemy as the ORM. database.py handles database configuration, session.py manages database sessions,

dependencies.py provides sessions to FastAPI endpoints, and models/complaint.py defines the Complaint database model."

BACKEND INTERVIEW QUESTIONS

1. WHAT IS FASTAPI AND WHY IS IT USED IN THIS PROJECT?

FastAPI is a Python web framework used to build REST APIs quickly and efficiently. In AIVOA, it provides the backend API through which the frontend communicates with the complaint and AI modules. It also provides request handling, validation support, routing, and automatic API documentation.

2. WHAT IS THE ROLE OF MAIN.PY IN A FASTAPI APPLICATION?

main.py is the entry point of the AIVOA backend application. It creates the FastAPI app, configures CORS, creates database tables, and includes the AI and complaint routers. It also defines the root endpoint of the application.

3. WHAT IS THE DIFFERENCE BETWEEN AN API ROUTE AND A SERVICE LAYER IN A BACKEND APPLICATION?

An API route receives HTTP requests and returns responses to the client. A service layer contains the actual business logic and keeps it separate from the API endpoints. In AIVOA, the API layer handles client requests while the AI and database logic are maintained separately.

4. WHAT ARE ROUTERS IN FASTAPI AND WHY ARE THEY SEPARATED INTO DIFFERENT FILES?

Routers group related API endpoints together. In AIVOA, AI endpoints and complaint endpoints are separated into different routers. This keeps the backend organized and makes the application easier to maintain as it grows.

5. HOW DOES FASTAPI HANDLE HTTP REQUESTS AND RESPONSES?

FastAPI maps an incoming HTTP request to the appropriate API route.
The route processes the request and returns a Python object that FastAPI converts into an HTTP response.
For example, AIVOA's root route returns a message confirming that the API is running.

6. WHAT IS DEPENDENCY INJECTION IN FASTAPI AND WHY IS IT USEFUL?

Dependency injection allows FastAPI to provide required resources to an API function automatically.
It is useful for managing shared resources such as database sessions without creating them repeatedly inside every route.
This improves code reuse and separation of responsibilities.

7. HOW IS DATABASE SESSION MANAGEMENT HANDLED IN A FASTAPI APPLICATION?

Database session management is handled through the database layer and can be provided to API routes using FastAPI dependencies.
The route uses the session for database operations and the session is then properly closed.
This prevents unnecessary database connections from remaining open.

8. WHAT IS THE PURPOSE OF SEPARATING DATABASE DEPENDENCIES FROM API ROUTES?

Database dependencies keep database session management separate from API business logic.
This prevents every API route from containing its own database connection code.
It makes the backend easier to test, maintain, and reuse.

9. WHAT IS PYDANTIC AND WHY IS IT USED IN THIS PROJECT?

Pydantic is a Python library used for data validation and structured data models.
In AIVOA, it is used to validate complaint and AI generated data before the application uses it.
For example, AIComplaintResponse is used to validate the structured complaint response from the LLM.

10. WHAT IS THE DIFFERENCE BETWEEN A PYDANTIC SCHEMA AND A SQLALCHEMY MODEL?

A Pydantic schema is mainly used to validate and structure API or application data.
A SQLAlchemy model represents a database table and is used for database operations.
In AIVOA, Pydantic handles structured complaint data while the SQLAlchemy Complaint model represents complaint data in the database.

1. HOW DOES REQUEST VALIDATION WORK IN FASTAPI USING PYDANTIC?

FastAPI uses Pydantic schemas to validate incoming request data before the API processes it. It checks the data type, required fields, and expected structure. If the request is invalid, FastAPI returns a validation error instead of processing incorrect data.

2. WHY SHOULD API RESPONSES ALSO BE VALIDATED USING SCHEMAS?

Response validation ensures that the backend returns data in the expected structure and format.

It prevents unexpected or incomplete data from reaching the frontend.

In AIVOA, schemas help ensure that complaint and AI response data has the expected structure.

3. WHAT IS SQLALCHEMY AND WHY IS IT USED INSTEAD OF WRITING RAW SQL QUERIES?

SQLAlchemy is a Python ORM library used to interact with the database using Python objects.

It reduces the need to write raw SQL for common database operations.

In AIVOA, SQLAlchemy is used to represent and manage complaint data through Python models.

4. WHAT IS ORM AND HOW DOES SQLALCHEMY IMPLEMENT ORM?

ORM stands for Object Relational Mapping.

It maps Python classes and objects to database tables and rows.

In AIVOA, the Complaint SQLAlchemy model represents complaint records in the database.

5. HOW DOES A SQLALCHEMY MODEL REPRESENT A DATABASE TABLE?

A SQLAlchemy model defines a Python class whose attributes represent database columns.

Each object of the class represents a row in the corresponding table.

For example, the AIVOA Complaint model represents the complaint table.

6. WHAT IS THE PURPOSE OF SEPARATING MODELS, SCHEMAS, AND API ROUTES?

Models handle database structure and database operations.

Schemas handle data validation and API data structure, while routes handle HTTP requests and responses.

This separation keeps the AIVOA backend modular and easier to maintain.

7. HOW DOES FASTAPI COMMUNICATE WITH THE DATABASE THROUGH SQLALCHEMY?

FastAPI receives the API request and uses a database session to perform database operations.

SQLAlchemy translates Python operations into database queries.

The result is then returned by the FastAPI endpoint.

8. WHAT IS THE PURPOSE OF A DATABASE SESSION AND HOW IS IT USED DURING AN API REQUEST?

A database session provides a connection context for performing database operations. During an API request, it can be used to query, insert, update, or delete complaint records. After the operation is completed, the session should be closed.

9. WHAT HAPPENS IF A DATABASE SESSION IS NOT PROPERLY CLOSED?

An unclosed session can keep database connections occupied unnecessarily. Over time, this can cause connection exhaustion and reduce application performance. In a production application, sessions should always be properly managed.

10. HOW WOULD YOU HANDLE AN INVALID COMPLAINT ID IN A COMPLAINT API?

The API should first check whether the complaint exists for the given ID. If no complaint is found, it should return an appropriate HTTP error such as 404 Not Found. This prevents the API from performing operations on a non-existing complaint.

1. HOW WOULD YOU HANDLE VALIDATION ERRORS COMING FROM A COMPLAINT REQUEST?

FastAPI automatically validates the request using Pydantic schemas before the request reaches the business logic.

If required fields are missing or values have an invalid type or format, FastAPI returns a validation error.

In AIVOA, this prevents invalid complaint data from being processed or stored in the database.

2. WHAT IS THE DIFFERENCE BETWEEN PUT AND PATCH WHEN UPDATING COMPLAINT DATA?

PUT is generally used to replace the complete resource with a new representation.

PATCH is used to update only specific fields of an existing resource.

For AIVOA complaint editing, PATCH is more suitable because a user may want to change only the batch number or quantity.

3. HOW WOULD YOU IMPLEMENT PARTIAL UPDATES FOR AN EXISTING COMPLAINT?

First, retrieve the existing complaint using its ID.

Then update only the fields provided in the request and keep all other existing values unchanged.

Finally, commit the updated complaint through the SQLAlchemy session and return the updated data.

4. HOW DOES THE BACKEND PREVENT EXISTING COMPLAINT INFORMATION FROM BEING OVERWRITTEN DURING AN EDIT?

The AI edit logic receives both the current complaint and the user's requested update. The prompt explicitly instructs the LLM to update only mentioned fields and preserve existing information.

The returned fields are then merged into the existing complaint rather than replacing the complete complaint. This behavior is implemented in `edit_complaint_node`.

5. HOW WOULD YOU HANDLE FILE UPLOADS IN FASTAPI?

FastAPI can receive uploaded files using its file upload support.

The backend should validate the file type and size, store the file safely, and extract its content when required.

In AIVOA, uploaded complaint documents are processed by the document extraction part of the AI layer.

6. WHY IS DOCUMENT PARSING SEPARATED INTO A UTILITY FILE INSTEAD OF KEEPING IT INSIDE THE API ROUTE?

Separating document parsing keeps file processing logic independent from API request handling.

This makes the API route smaller and allows the parser to be reused by different parts of the application.

It also makes testing and future support for additional document formats easier.

7. HOW SHOULD A BACKEND HANDLE UNSUPPORTED UPLOADED FILE TYPES?

The backend should validate the file extension and preferably the actual content type before processing it.

If the format is unsupported, it should reject the request with an appropriate HTTP error.

This prevents the document parser and AI layer from receiving files they cannot process.

8. HOW WOULD YOU PREVENT SENSITIVE CONFIGURATION SUCH AS API KEYS FROM BEING HARDCODED IN THE BACKEND?

Sensitive values should be stored in environment variables rather than directly inside Python files.

The application configuration layer can load these values when the application starts.

In AIVOA, the Groq API key is accessed through `settings.groq_api_key` rather than being written directly in `llm.py`.

9. WHAT IS THE PURPOSE OF AN ENVIRONMENT CONFIGURATION FILE IN A BACKEND APPLICATION?

An environment configuration file stores values that can change between environments, such as API keys and database credentials.

It keeps sensitive configuration separate from application logic.

This allows the same codebase to run in development, testing, and production with different configurations.

10. HOW WOULD YOU HANDLE EXCEPTIONS OCCURRING DURING DATABASE OPERATIONS?

Database operations should be placed inside appropriate exception handling logic. If an operation fails, the transaction should be rolled back and the error should be logged. The API should then return a controlled HTTP error instead of exposing internal database details to the client.

11. HOW WOULD YOU DESIGN AN API SO THAT AI PROCESSING ERRORS DO NOT CRASH THE ENTIRE BACKEND?

AI processing should be isolated behind controlled API endpoints and exception handling. If the LLM fails, returns invalid JSON, or becomes unavailable, the backend should catch the error and return an appropriate response.

This allows the FastAPI server to remain available even when the AI service fails.

12. WHY IS IT USEFUL TO SEPARATE AI APIS FROM COMPLAINT MANAGEMENT APIS?

AI APIs handle intelligent operations such as complaint extraction and document processing. Complaint APIs handle normal complaint management and database operations. Separating them keeps responsibilities clear and makes the system easier to maintain and extend.

13. HOW WOULD YOU MAKE THE BACKEND EASIER TO MAINTAIN AS THE NUMBER OF API ENDPOINTS INCREASES?

Related endpoints should be grouped into separate routers instead of keeping everything in one file.

Business logic, database logic, schemas, and utilities should also remain separated.

AIVOA already follows this structure by separating AI and complaint routers in main.py.

14. HOW WOULD YOU SECURE BACKEND APIS SO THAT UNAUTHORIZED USERS CANNOT ACCESS COMPLAINT DATA?

Authentication should verify the identity of the user before allowing access to protected endpoints.

Authorization should then check whether that user has permission to access or modify the requested complaint.

Sensitive data should also be protected using secure authentication tokens and proper server-side validation.

15. HOW WOULD YOU IMPROVE THE SCALABILITY AND MAINTAINABILITY OF THIS FASTAPI BACKEND IF THE PROJECT GROWS SIGNIFICANTLY?

I would further separate business logic into service modules and keep API routes focused on request handling.

For scalability, I would add proper authentication, database connection pooling, caching

where useful, background processing for heavy AI tasks, logging, monitoring, and automated testing.

The existing separation of API, AI, database, models, schemas, and utilities provides a good foundation for these improvements.

DATABASE

1. WHAT IS THE PURPOSE OF USING MYSQL IN THIS PROJECT?

MySQL is used as the relational database for storing AIVOA complaint data permanently. It allows the backend to store, retrieve, update, and manage structured complaint records. FastAPI connects to the database through SQLAlchemy.

2. WHAT IS THE ROLE OF SQLALCHEMY BETWEEN THE FASTAPI BACKEND AND MYSQL DATABASE?

SQLAlchemy acts as the ORM layer between FastAPI and MySQL. It allows the application to work with Python models and database sessions instead of writing raw SQL for every operation. In AIVOA, `create_engine` and `sessionmaker` are used to configure database access.

3. WHAT IS THE DIFFERENCE BETWEEN A DATABASE MODEL AND A PYDANTIC SCHEMA IN THIS PROJECT?

A database model represents the structure of data stored in the MySQL database. A Pydantic schema validates and structures data entering or leaving the API. In AIVOA, the Complaint model represents database complaint records, while Pydantic schemas handle API and AI data validation.

4. WHAT IS THE PURPOSE OF THE COMPLAINT MODEL AND WHAT TYPE OF DATA DOES IT REPRESENT?

The Complaint model represents complaint records stored in the database. It contains structured information related to a pharmaceutical complaint, such as complaint details, product information, and complaint status. SQLAlchemy uses this model to map Python objects to database records.

5. HOW WOULD YOU DESIGN THE COMPLAINT TABLE FOR STORING ALL COMPLAINT INFORMATION?

I would divide the complaint information into appropriate columns based on the complaint requirements.

Important fields would include complaint identification, customer information, product information, batch details, complaint description, severity, priority, and timestamps.

Fields that may contain multiple independent records should be separated into related tables rather than storing everything in one column.

6. WHAT IS A PRIMARY KEY AND WHICH FIELD WOULD YOU USE AS THE PRIMARY KEY FOR COMPLAINTS?

A primary key uniquely identifies each record in a database table.

For the Complaint table, I would use a dedicated numeric ID as the primary key.

The complaint_number can remain a separate business identifier.

7. WHY SHOULD COMPLAINT_NUMBER BE DIFFERENT FROM THE DATABASE PRIMARY KEY?

The primary key is mainly used internally by the database to uniquely identify a record.

complaint_number is a business-level identifier that can be shown to users and used for complaint tracking.

Keeping them separate allows the internal database ID and business identifier to have different purposes.

8. WHAT IS THE PURPOSE OF CREATED_AT AND UPDATED_AT FIELDS IN THE COMPLAINT TABLE?

created_at records when the complaint was originally created.

updated_at records when the complaint was last modified.

These fields provide an audit trail of when the complaint record was created and changed.

9. WHAT IS THE DIFFERENCE BETWEEN NULLABLE AND NON-NULLABLE COLUMNS IN A COMPLAINT TABLE?

A nullable column can contain NULL when information is not available.

A non-nullable column must contain a value when the record is created.

For example, required complaint identification data should generally be non-nullable, while optional complaint details can be nullable.

10. HOW WOULD YOU ENFORCE UNIQUENESS FOR FIELDS SUCH AS COMPLAINT_NUMBER?

I would add a unique constraint or unique index to the complaint_number column.

This makes the database reject duplicate complaint numbers.

It provides database-level protection even if duplicate data is accidentally sent by the application.

1. HOW WOULD YOU ENFORCE UNIQUENESS FOR FIELDS SUCH AS COMPLAINT_NUMBER?

I would use a UNIQUE constraint on the complaint_number column.

This makes sure that two complaint records cannot have the same complaint number.

The database itself enforces this rule, even if duplicate data is sent by the application.

2. WHAT IS A FOREIGN KEY AND WHERE WOULD IT BE USEFUL IN THIS PROJECT?

A foreign key connects a column in one table to the primary key of another table.

In AIVOA, it could connect a complaint to a user, attachment, or related record.

It helps maintain relationships and prevents invalid references between tables.

3. HOW WOULD YOU ESTABLISH RELATIONSHIPS BETWEEN USERS, COMPLAINTS, ATTACHMENTS, AND CHAT MESSAGES?

I would use primary keys and foreign keys to establish the relationships.

A user can have multiple complaints, while a complaint can have multiple attachments and chat messages.

For example, user_id can be referenced in the complaint table, and complaint_id can be referenced by attachments and messages.

4. WHAT IS NORMALIZATION AND WHY IS IT IMPORTANT WHEN DESIGNING THIS PROJECT'S DATABASE?

Normalization is the process of organizing data into related tables to reduce unnecessary duplication.

For AIVOA, user, complaint, attachment, and chat data can be separated based on their responsibilities.

This improves data consistency and makes updates easier.

5. WHAT PROBLEMS CAN OCCUR IF COMPLAINT DATA IS STORED IN A SINGLE UNSTRUCTURED TABLE?

It can cause duplicate data, inconsistent values, and difficult updates.

It can also create large tables with repeated information and make relationships harder to manage.

As AIVOA grows, querying and maintaining such a structure would become more difficult.

6. WHAT INDEXES WOULD YOU CREATE FOR THE COMPLAINT TABLE AND WHY?

I would consider indexes on fields frequently used for searching and filtering, such as complaint_number, batch_lot_number, complaint_date, severity, and status if those fields exist in the final model.

The exact indexes should be based on actual query patterns.

Indexes can improve read performance but also add storage and write overhead.

7. HOW WOULD INDEXING AFFECT COMPLAINT SEARCH AND RETRIEVAL PERFORMANCE?

An appropriate index allows the database to locate matching complaint records without scanning the entire table.

For example, searching by complaint_number can become much faster with an index on that column.

However, too many indexes can slow INSERT and UPDATE operations.

8. WHAT IS THE DIFFERENCE BETWEEN DELETE AND TRUNCATE AND WHEN WOULD EACH BE USED?

DELETE removes selected rows and can use a WHERE condition.

TRUNCATE removes all rows from a table and is generally used when the entire table needs to be cleared.

For production complaint data, DELETE would normally be safer because individual records can be targeted.

9. HOW WOULD YOU MAINTAIN DATA INTEGRITY WHEN UPDATING OR DELETING A COMPLAINT?

I would use database constraints, foreign keys, transactions, and proper validation. Before updating or deleting, the backend should verify that the complaint exists and that the operation is allowed.

Transactions help ensure that a failed operation does not leave partially updated data.

10. HOW WOULD YOU HANDLE CONCURRENT USERS UPDATING THE SAME COMPLAINT?

I would use transaction control and an appropriate concurrency strategy.

For example, optimistic locking can use a version or updated_at value to detect whether another user changed the complaint before the current update.

If a conflict is detected, the backend can reject the update and ask the user to reload the latest data.

11. HOW WOULD YOU OPTIMIZE THE DATABASE IF THE NUMBER OF COMPLAINTS GROWS FROM THOUSANDS TO MILLIONS?

I would first analyze slow queries and add indexes based on actual access patterns.

I would also use pagination, connection pooling, query optimization, and appropriate database constraints.

For very large datasets, partitioning or archiving older complaint records could also be considered.

AI AIVOA

1. WHAT IS THE PURPOSE OF THE AI LAYER IN THIS PROJECT?

The AI layer processes user complaint information using an LLM.

It handles intent detection, complaint extraction, complaint editing, document extraction, and risk assessment.

In AIVOA, this layer converts unstructured user input into structured complaint information.

2. WHY IS LANGGRAPH USED INSTEAD OF IMPLEMENTING THE AI WORKFLOW AS A SINGLE LLM CALL?

LangGraph allows the AI process to be divided into separate nodes with specific responsibilities.

AIVOA can route different requests to different operations such as logging, editing, or document extraction.

This makes the AI workflow more structured and easier to control.

3. WHAT IS LANGGRAPH AND WHAT PROBLEM DOES IT SOLVE IN THIS PROJECT?

LangGraph is a framework for building stateful AI workflows using connected nodes. In AIVOA, it manages the sequence and routing of different AI operations. It also allows information to be shared between nodes through ComplaintState.

4. WHAT IS THE PURPOSE OF COMPLAINTSTATE IN THE LANGGRAPH WORKFLOW?

ComplaintState stores the information shared between different AI nodes. It allows one node to produce information that another node can use. For example, detected intent and complaint data can be stored in the shared state.

5. WHAT INFORMATION IS STORED INSIDE COMPLAINTSTATE?

ComplaintState contains `user_message`, `uploaded_document`, `intent`, `complaint`, `assistant_response`, and `error`. These fields represent the input, detected intent, extracted complaint information, response, and possible error. The state is defined using TypedDict.

6. WHAT IS THE ROLE OF GRAPH.PY IN THE AI ARCHITECTURE?

`graph.py` defines the LangGraph workflow and connects the AI processing nodes. It determines how the request moves between intent detection, complaint processing, document extraction, and risk assessment. It also compiles the graph into an executable workflow.

7. WHAT IS THE PURPOSE OF ROUTE_INTENT(STATE: COMPLAINTSTATE)?

`route_intent` checks the value of `state["intent"]`. It returns the appropriate next operation based on the detected intent. For example, it can route the request toward complaint logging, editing, or document extraction.

8. HOW DOES THE AI LAYER DETERMINE WHETHER THE USER WANTS TO LOG, EDIT, OR EXTRACT A COMPLAINT?

The `detect_intent_node` sends the user's message to the LLM with the intent classification prompt. The LLM identifies the appropriate intent from the supported operations. The detected intent is then stored in the workflow state.

9. WHAT IS THE PURPOSE OF DETECT_INTENT_NODE(STATE: COMPLAINTSTATE)?

`detect_intent_node` identifies what operation the user is requesting. It uses the LLM to classify the request into the supported complaint operations. The result is stored in `state["intent"]`.

10. WHAT IS THE DIFFERENCE BETWEEN INTENT DETECTION AND COMPLAINT EXTRACTION?

Intent detection determines what the user wants to do.

Complaint extraction identifies the actual complaint information from the user's input. For example, intent detection identifies "log complaint", while extraction identifies the product, batch, quantity, and complaint details.

11. WHAT IS THE PURPOSE OF LOG_COMPLAINT_NODE(STATE: COMPLAINTSTATE)?

log_complaint_node processes a new complaint provided by the user.

It uses the LLM to extract structured complaint information from the input.

It then stores the complaint and related risk assessment information in the state.

12. HOW DOES THE AI CONVERT UNSTRUCTURED COMPLAINT TEXT INTO STRUCTURED COMPLAINT DATA?

The user's natural language complaint is provided to the LLM with a defined system prompt.

The LLM extracts the relevant information and returns it in the expected structured format.

The response is then parsed and validated before being stored as complaint data.

13. WHAT IS THE PURPOSE OF EDIT_COMPLAINT_NODE(STATE: COMPLAINTSTATE)?

edit_complaint_node handles changes requested for an existing complaint.

It identifies which complaint fields the user wants to modify.

Only the requested information is changed while the remaining complaint information is preserved.

14. HOW DOES THE SYSTEM PERFORM A PARTIAL COMPLAINT UPDATE WITHOUT OVERWRITING EXISTING INFORMATION?

The existing complaint data is provided along with the user's requested change.

The AI identifies only the fields that need modification.

The updated values are then merged with the existing complaint instead of replacing the complete complaint.

15. WHAT IS THE PURPOSE OF DOCUMENT_EXTRACTION_NODE(STATE: COMPLAINTSTATE)?

document_extraction_node processes complaint information contained in an uploaded document.

It sends the document content to the LLM for structured complaint extraction.

The extracted complaint can then be used for further AI processing.

16. HOW DOES THE AI LAYER PROCESS COMPLAINT INFORMATION EXTRACTED FROM AN UPLOADED DOCUMENT?

The uploaded document is first converted into readable content.

That content is passed to the document extraction AI node.

The LLM extracts the complaint information and returns it in the required structured format.

17. WHAT IS THE PURPOSE OF RISK_ASSESSMENT_NODE(STATE: COMPLAINTSTATE)?

risk_assessment_node generates a risk assessment for the complaint.

It calls generate_risk_assessment using the complaint data.

The generated assessment is then stored in the AI workflow state.

18. WHAT IS THE ROLE OF GENERATE_RISK_ASSESSMENT(COMPLAINT: DICT)?

generate_risk_assessment sends the complaint data to the Groq LLM with the risk assessment prompt.

The LLM analyzes the complaint and returns a structured JSON risk assessment.

The JSON response is then converted into a Python dictionary.

19. WHAT INFORMATION IS GENERATED BY THE RISK ASSESSMENT FUNCTION?

It generates severity, next_action, justification, complaint_summary, root_cause_recommendation, and capa_recommendation.

It also generates completeness_check and duplicate_complaint_check.

These fields provide the QA related assessment of the complaint.

20. HOW DOES THE SYSTEM DETERMINE THE SEVERITY OF A COMPLAINT?

The risk assessment prompt instructs the LLM to classify severity as Critical, High, Medium, or Low.

The LLM uses the provided complaint data to select the appropriate severity.

The system explicitly instructs the LLM to use only the given complaint data and not invent information.

1. WHAT IS CAPA AND WHY IS CAPA RECOMMENDATION INCLUDED IN THE RISK ASSESSMENT?

CAPA means Corrective and Preventive Action.

It is included to recommend actions for correcting the current complaint issue and preventing similar issues in the future.

In AIVOA, capa_recommendation is one of the fields generated by the risk assessment.

2. HOW DOES THE SYSTEM PERFORM A DUPLICATE COMPLAINT CHECK?

The current AI prompt includes duplicate_complaint_check as a required risk assessment field.

It instructs the AI to mention that a duplicate check should be performed.

The provided implementation does not show an actual database based duplicate matching algorithm.

3. WHAT IS THE PURPOSE OF COMPLETENESS_CHECK IN THE RISK ASSESSMENT?

completeness_check identifies important complaint information that may be missing.

It helps QA determine whether enough information is available for further investigation.

The field is generated as part of the AI risk assessment.

4. WHY IS RISK ASSESSMENT SEPARATED FROM COMPLAINT EXTRACTION?

Complaint extraction focuses on identifying and structuring complaint information.
Risk assessment focuses on analyzing that complaint and recommending the next QA action.
Separating them makes each AI operation easier to maintain and modify independently.

5. WHAT IS THE PURPOSE OF `_PARSE_AND_VALIDATE(CONTENT: STR)`?

`_parse_and_validate` processes the raw LLM response before it is used by the application.
It cleans the response, converts it into structured data, and validates it against the expected schema.

This helps prevent incorrectly formatted AI output from entering the application.

6. WHY IS VALIDATION REQUIRED FOR AN LLM GENERATED RESPONSE?

LLMs can return unexpected structures or invalid data even when given clear instructions.
Validation ensures the response follows the application's expected format.
In AIVOA, structured validation helps keep complaint and risk assessment data reliable.

7. HOW DOES THE SYSTEM HANDLE AN LLM RESPONSE WRAPPED INSIDE MARKDOWN CODE FENCES?

The system checks whether the response starts with a Markdown code fence.
It removes `json` and before parsing the response.
This allows the JSON content to be processed normally.

8. WHAT HAPPENS IF THE LLM RETURNS INVALID JSON?

The current implementation uses `json.loads(content)` to parse the response.
If the content is invalid JSON, parsing raises an exception.
The provided code does not show a separate recovery mechanism for this error.

9. WHAT IS THE PURPOSE OF `COMPLAINTEXTRACTIONOUTPUT`?

`ComplaintExtractionOutput` defines the expected structure for complaint extraction.
It contains a `complaint` field of type `dict`.
This provides a structured format for extracted complaint information.

10. WHAT IS THE DIFFERENCE BETWEEN `COMPLAINTEXTRACTIONOUTPUT` AND `AICOMPLAINTRESPONSE`?

`ComplaintExtractionOutput` focuses on the extracted complaint structure.
`AIComplaintResponse` is used for validating the broader AI response containing complaint and risk assessment information.
So, one focuses on extraction while the other validates the complete AI response.

11. WHAT IS THE PURPOSE OF `INTENT_CLASSIFIER_SYSTEM_PROMPT`?

`INTENT_CLASSIFIER_SYSTEM_PROMPT` provides instructions to the LLM for identifying the user's requested operation.
It helps classify the request into the supported AI operations.
This classification is then used for routing the workflow.

12. WHAT IS THE PURPOSE OF LOG_COMPLAINT_SYSTEM_PROMPT?

LOG_COMPLAINT_SYSTEM_PROMPT provides the rules for complaint extraction, editing, and risk assessment.

It defines what information the LLM should extract and how the output should be structured.

It also instructs the AI to avoid inventing information.

13. WHY ARE PROMPTS SEPARATED INTO A DEDICATED PROMPTS.PY FILE?

Keeping prompts in prompts.py separates AI instructions from the processing logic.

It makes prompts easier to update without modifying the AI node functions.

It also keeps the nodes.py implementation cleaner.

14. WHY IS TEMPERATURE SET TO 0 FOR THE LLM IN THIS PROJECT?

Temperature 0 makes the model's responses more deterministic and consistent.

This is useful because AIVOA expects structured complaint and risk assessment output.

It reduces unnecessary variation in repeated requests.

15. WHAT IS THE PURPOSE OF BASE_LLM IN LLM.PY?

base_llm is the configured Groq ChatGroq model instance used by the AI layer.

The AI functions use this same LLM instance to process intent, complaints, and risk assessments.

The configured model is llama-3.1-8b-instant with temperature 0.

16. WHY IS STRUCTURED OUTPUT IMPORTANT WHEN USING AN LLM FOR COMPLAINT PROCESSING?

Structured output makes AI responses predictable and easier for the backend to process.

Instead of receiving free-form text, the application receives defined fields such as complaint and risk assessment.

This is important because the data may later be stored or used by other backend components.

17. HOW DOES THE AI LAYER HANDLE UPLOADED DOCUMENTS DIFFERENTLY FROM DIRECT USER MESSAGES?

Direct user messages are processed as user input for intent detection or complaint extraction.

Uploaded documents are first converted into readable content and then passed to the document extraction process.

The AI then extracts structured complaint information from that document content.

18. HOW WOULD YOU HANDLE HALLUCINATIONS OR INCORRECT INFORMATION GENERATED BY THE LLM?

I would restrict the prompt to use only the provided complaint information and explicitly prevent the model from inventing details.

I would validate the output using schemas and reject invalid structures.
For production, important AI decisions should also have human QA review and verification.

19. HOW WOULD YOU IMPROVE THE AI WORKFLOW IF THE NUMBER OF COMPLAINT TYPES AND AI OPERATIONS INCREASES?

I would add separate LangGraph nodes for new operations instead of putting everything into one function.

The routing logic could then direct each intent to its corresponding node.

This keeps individual operations modular and makes the workflow easier to extend.

20. HOW WOULD YOU IMPROVE THE RELIABILITY AND ACCURACY OF THE AI LAYER FOR PRODUCTION USE?

I would improve prompt design, add stronger schema validation, test the system with real complaint examples, and monitor incorrect AI responses.

I would also add retry and fallback handling for LLM failures and invalid JSON.

For high impact decisions such as risk classification, I would include human QA verification.